

EcoTrack: Smart City Waste Management System

DBMS Course Project Report

Date: April 2026

Subject: Database Management Systems (4th Sem)

1. Abstract

EcoTrack is an integrated software solution designed to modernize urban waste management. By leveraging a relational database (MSSQL) and a real-time web interface (Node.js), the system optimizes collection routes, incentivizes citizen participation through “Green Points,” and provides administrators with data-driven insights into city-wide waste production.

2. Problem Statement & Objective

2.1 The Problem

Traditional waste management systems are often static and inefficient. Bins are collected on a fixed schedule regardless of whether they are full or empty, leading to wasted fuel and overflowing bins in high-traffic areas. Furthermore, there is no mechanism to encourage citizens to report waste issues.

2.2 Project Objectives

- **Dynamic Monitoring:** Track bin levels (Normal vs. Overflowing) in real-time.
 - **Incentivization:** Implement a rewards system to encourage reporting.
 - **Data Analytics:** Analyze waste production trends by zone and material type.
 - **Role-Based Access:** Provide customized interfaces for Citizens, Drivers, and Admins.
-

3. System Architecture

The system follows a **Three-Tier Architecture**:

1. Presentation Tier (Frontend):

- Built with HTML5, CSS3, and JavaScript.
- Utilizes **Chart.js** for real-time data visualization.
- Responsive dark-themed UI for mobile and desktop use.

- 2. **Logic Tier (Backend):**
 - **Node.js & Express.js:** Handles API routing and business logic.
 - **RESTful API:** Facilitates communication between the UI and the database.
 - **Authentication:** Role-based logic to verify user permissions.
 - 3. **Data Tier (Database):**
 - **Microsoft SQL Server (MSSQL):** Stores all relational data.
 - **Tedious Driver:** Ensures cross-platform compatibility (Linux/Windows).
-

4. Database Design (DBMS Focus)

4.1 Entity Relationship (ER) Summary

- **Users:** (PK: id, username, password, role, green_points)
- **Bins:** (PK: id, location, zone, status, level)
- **Requests:** (PK: id, FK: user_id, type, details, status)
- **WasteLogs:** (PK: id, waste_type, weight, logged_at)

4.2 DBMS Concepts Applied

- **Normalization:** The database is designed in **3rd Normal Form (3NF)** to eliminate redundancy.
 - **Referential Integrity:** Enforced via Foreign Keys (e.g., `Requests.user_id` links to `Users.id`).
 - **Triggers:** Automated logic updates the `Requests` status to 'Resolved' when a completion timestamp is added.
 - **Stored Procedures:** Used for complex operations like adding requests while simultaneously updating user points (`sp_AddRequest`).
 - **Views:** Virtual tables like `vw_CitizenLeaderboard` and `vw_WasteAnalytics` simplify complex JOIN and AGGREGATE queries for the frontend.
-

5. Feature Breakdown

5.1 Citizen Portal

- **Feature:** Report overflowing bins or request special pickups.
- **Impact:** Every valid report awards the user **10 Green Points**, gamifying city cleanliness.

5.2 Driver Portal

- **Feature:** Real-time task list.
- **Impact:** Drivers see a prioritized list where **Overflowing Bins** appear at the top, ensuring critical issues are handled first.

5.3 Admin Dashboard

- **Feature:** Analytics and User Management.
 - **Impact:** Displays charts showing the distribution of waste types (Plastic, Glass, Metal, Bio) and zone-wise collection efficiency.
-

6. Technology Stack & Tools

Component	Tool Used
Backend Environment	Node.js (LTS)
Web Framework	Express.js
Database System	MS SQL Server 2022
Driver (Linux/Mac/Win)	Tedious / mssql
Frontend Styling	Vanilla CSS (Flexbox & Grid)
Charts & Graphs	Chart.js
Configuration	Dotenv (Security)
Process Management	Docker (for MSSQL)

7. Setup & Installation Guide

7.1 Database Setup (Docker)

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=YourPassword123!" \  
-p 1433:1433 --name sql_server -d \  
mcr.microsoft.com/mssql/server:2022-latest
```

7.2 Application Setup

1. **Clone & Install:** `npm install`
2. **Environment:** Create `.env` from `.env.example`.
3. **Initialize DB:** `node init_db.js`
4. **Run Server:** `node server.js`

7.3 Containerization Strategy (Why Docker?)

The project utilizes Docker for the database layer to solve cross-platform compatibility issues. Microsoft SQL Server is natively designed for Windows; Docker allows it to run seamlessly on Linux or macOS by encapsulating the database within a lightweight, perfectly configured virtual container. This keeps the host system clean and ensures a plug-and-play setup for any developer.

7.4 Disaster Recovery & Multiple Instances

Because the database runs inside an isolated container, creating multiple, independent database servers is trivial. If the primary database fails or data is corrupted, a fresh backup instance can be spun up immediately by altering the port mapping and container name:

```
# Creating a secondary backup server on port 1434
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=YourPassword123!" \
  -p 1434:1433 --name sql_server_backup -d \
  mcr.microsoft.com/mssql/server:2022-latest
```

To permanently remove a broken primary container and free up the default port (1433):

```
docker stop sql_server
docker rm sql_server
```

8. Conclusion

EcoTrack demonstrates how modern database management concepts can be applied to solve real-world urban challenges. By centralizing waste data and providing role-specific tools, the system increases efficiency and encourages civic responsibility.

Instructions for PDF Generation

To convert this report into a professional PDF: 1. **VS Code:** Use the **Markdown PDF** extension (Right-click > Export PDF). 2. **LaTeX (Pandoc):** Run the following in your terminal: `bash pandoc DETAILED_REPORT.md -o EcoTrack_Final_Report.pdf --pdf-engine=xelatex` 3. **Online:** Copy this text into Dillinger.io and select **Export > PDF**.